CSA0604-DAA

Session 1 Lab Experinments

Name : K.YOGANAND

Reg.No: 192525173

1.Given an array of strings words, return the first palindromic string in the array. If there

is no such string, return an empty string "". A string is palindromic if it reads the same forward and backward.

Example 1:

Input: words = ["abc","car","ada","racecar","cool"]

Output: "ada"

Explanation: The first string that is palindromic is "ada".

Note that "racecar" is also palindromic, but it is not the first.

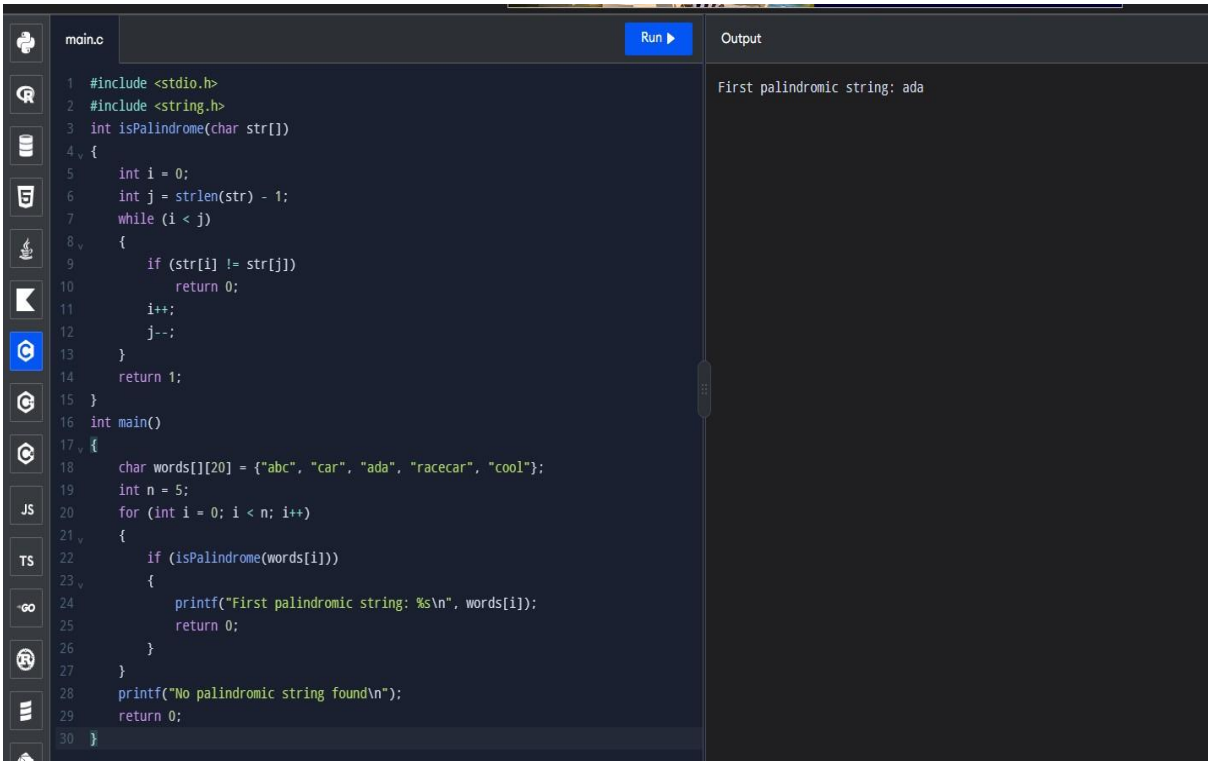
Example 2:

Input: words = ["notapalindrome","racecar"]

Output: "racecar"

Explanation: The first and only string that is palindromic is "racecar".

Output:



2.You are given two integer arrays nums1 and nums2 of sizes n and m, respectively.

Calculate the following values: answer1 : the number of indices i such that nums1[i] exists in nums2. answer2 : the number of indices i such that nums2[i] exists in nums1

Return [answer1,answer2].

Example 1:

Input: nums1 = [2,3,2], nums2 = [1,2]

Output: [2,1]

Explanation:

Example 2:

Input: nums1 = [4,3,2,3,1], nums2 = [2,2,5,2,3,6]

Output: [3,4]

Explanation:

The elements at indices 1, 2, and 3 in nums1 exist in nums2 as well. So answer1 is 3.

The elements at indices 0, 1, 3, and 4 in nums2 exist in nums1. So answer2 is 4.

OUTPUT:

```c
#include <stdio.h>
int main()
{
    int nums1[] = {2, 3, 2};
    int nums2[] = {1, 2};
    int n = 3;
    int m = 2;
    int answer1 = 0;
    int answer2 = 0;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            if (nums1[i] == nums2[j])
            {
                answer1++;
                break;
            }
        }
    }

    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (nums2[i] == nums1[j])
            {
                answer2++;
                break;
            }
        }
    }
    printf("[%d, %d]\n", answer1, answer2);
    return 0;
}
```

```
[2, 1]
```

3.You are given a 0-indexed integer array nums. The distinct count of a subarray of nums is defined as: Let nums[i..j] be a subarray of nums consisting of all the indices from i to j such that 0 <= i <= j < nums.length. Then the number of distinct values in nums[i..j] is called the distinct count of nums[i..j]. Return the sum of the squares of

distinct counts of all subarrays of nums. A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

Input: nums = [1,2,1]

Output: 15

Explanation: Six possible subarrays are:

[1]: 1 distinct value

[2]: 1 distinct value

[1]: 1 distinct value

[1,2]: 2 distinct values

[2,1]: 2 distinct values

[1,2,1]: 2 distinct values

The sum of the squares of the distinct counts in all subarrays is equal to 12 + 12 + 12 + 22 + 22 + 22 = 15.

Example 2:

Input: nums = [1,1]

Output: 3

Explanation: Three possible subarrays are:

[1]: 1 distinct value

[1]: 1 distinct value

[1,1]: 1 distinct value

The sum of the squares of the distinct counts in all subarrays is equal to 12 + 12 + 12 = 3.

OUTPUT:

main.c

Run

```c
#include <stdio.h>

int main()
{
    int nums[] = {1, 2, 1};
    int n = 3;
    int sum = 0;
    for (int i = 0; i < n; i++)
    {
        int visited[100] = {0};
        int distinct = 0;
        for (int j = i; j < n; j++)
        {
            if (visited[nums[j]] == 0)
            {
                visited[nums[j]] = 1;
                distinct++;
            }
            sum = sum + (distinct * distinct);
        }
    }
    printf("Sum of squares of distinct counts = %d\n", sum);
    return 0;
}
```

Output

```
Sum of squares of distinct counts = 15
```

4. Given a 0-indexed integer array nums of length n and an integer k, return the number of pairs (i, j) where 0 <= i < j < n, such that nums[i] == nums[j] and (i * j) is divisible by k.

Example 1:

Input: nums = [3,1,2,2,2,1,3], k = 2

Output: 4

Explanation:

There are 4 pairs that meet all the requirements:

- nums[0] == nums[6], and 0 * 6 == 0, which is divisible by 2.
- nums[2] == nums[3], and 2 * 3 == 6, which is divisible by 2.
- nums[2] == nums[4], and 2 * 4 == 8, which is divisible by 2.
- nums[3] == nums[4], and 3 * 4 == 12, which is divisible by 2.

Example 2:

Input: nums = [1,2,3,4], k = 1

Output: 0

Explanation: Since no value in nums is repeated, there are no pairs (i,j) that meet all the requirements.

OUTPUT:



5. Write a program FOR THE BELOW TEST CASES with least time complexity

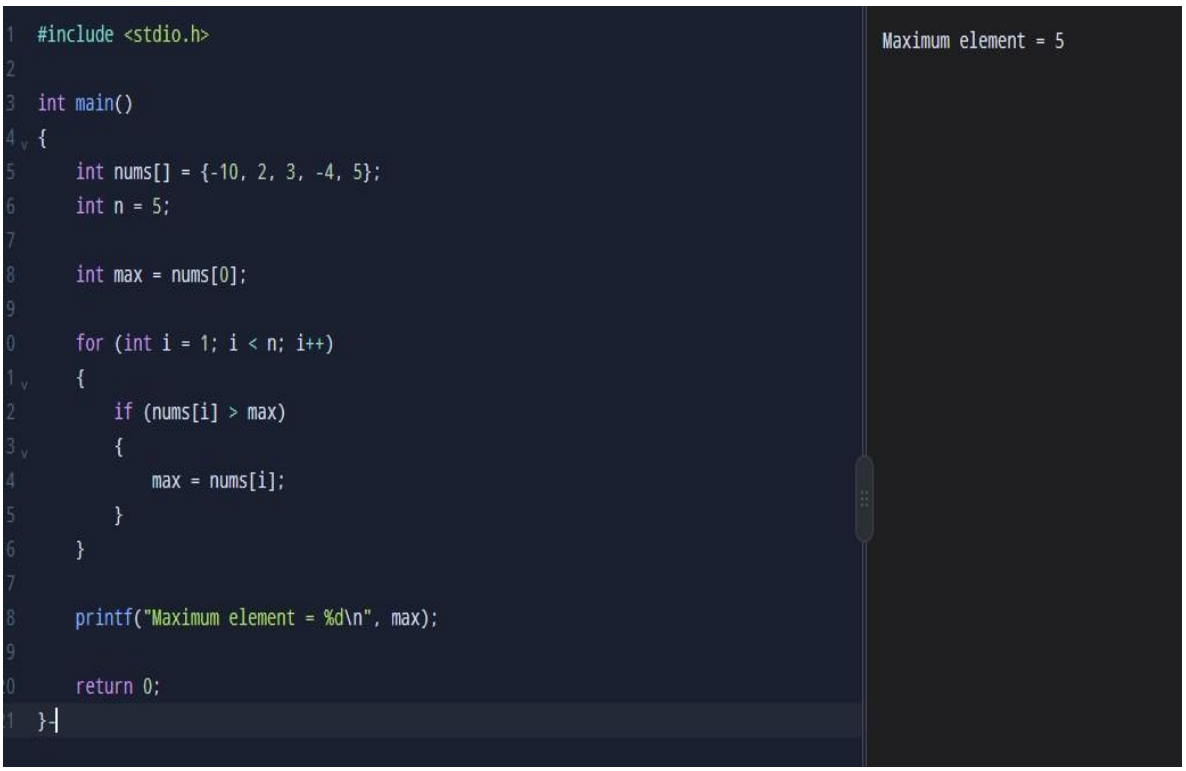
Test Cases: -

Input: {1, 2, 3, 4, 5} Expected Output: 5

Input: {7, 7, 7, 7, 7} Expected Output: 7

Input: {-10, 2, 3, -4, 5} Expected Output: 5

OUTPUT:



```c
#include <stdio.h>

int main()
{
    int nums[] = {-10, 2, 3, -4, 5};
    int n = 5;

    int max = nums[0];

    for (int i = 1; i < n; i++)
    {
        if (nums[i] > max)
        {
            max = nums[i];
        }
    }

    printf("Maximum element = %d\n", max);

    return 0;
}
```

```
Maximum element = 5
```

6. You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same. Test Cases

1. Empty List

1. Input: []
2. Expected Output: None or an appropriate message indicating that the list is empty.

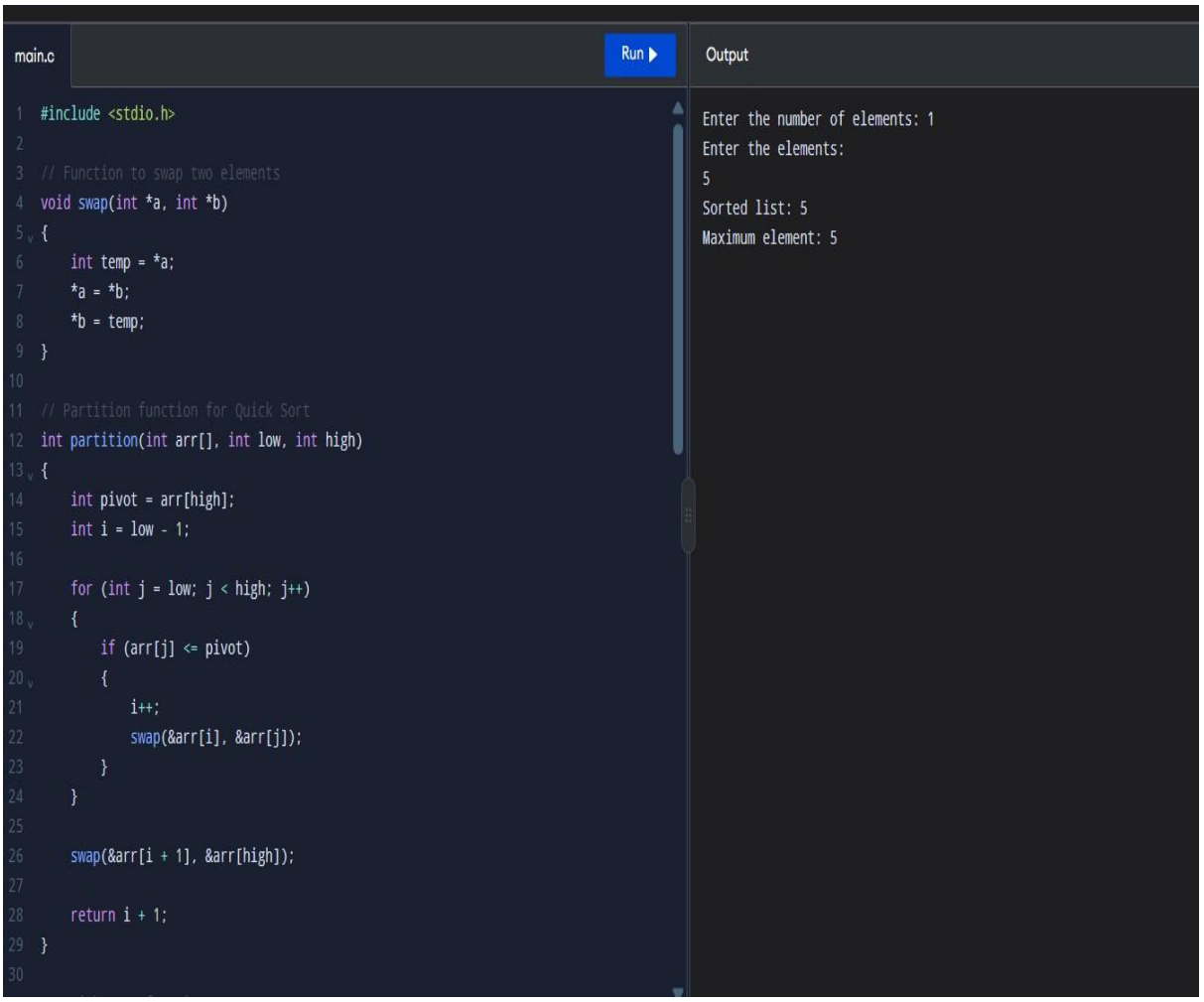
2. Single Element List

1. Input: [5]
2. Expected Output: 5

3. All Elements are the Same

1. Input: [3, 3, 3, 3, 3]
2. Expected Output: 3

OUTPUT:



7. Write a program that takes an input list of n numbers and creates a new list containing only the unique elements from the original list. What is the space complexity of the algorithm?

Test Cases

Some Duplicate Elements

Input: [3, 7, 3, 5, 2, 5, 9, 2]

Expected Output: [3, 7, 5, 2, 9] (Order may vary based on the algorithm used)

Negative and Positive Numbers

Input: [-1, 2, -1, 3, 2, -2]

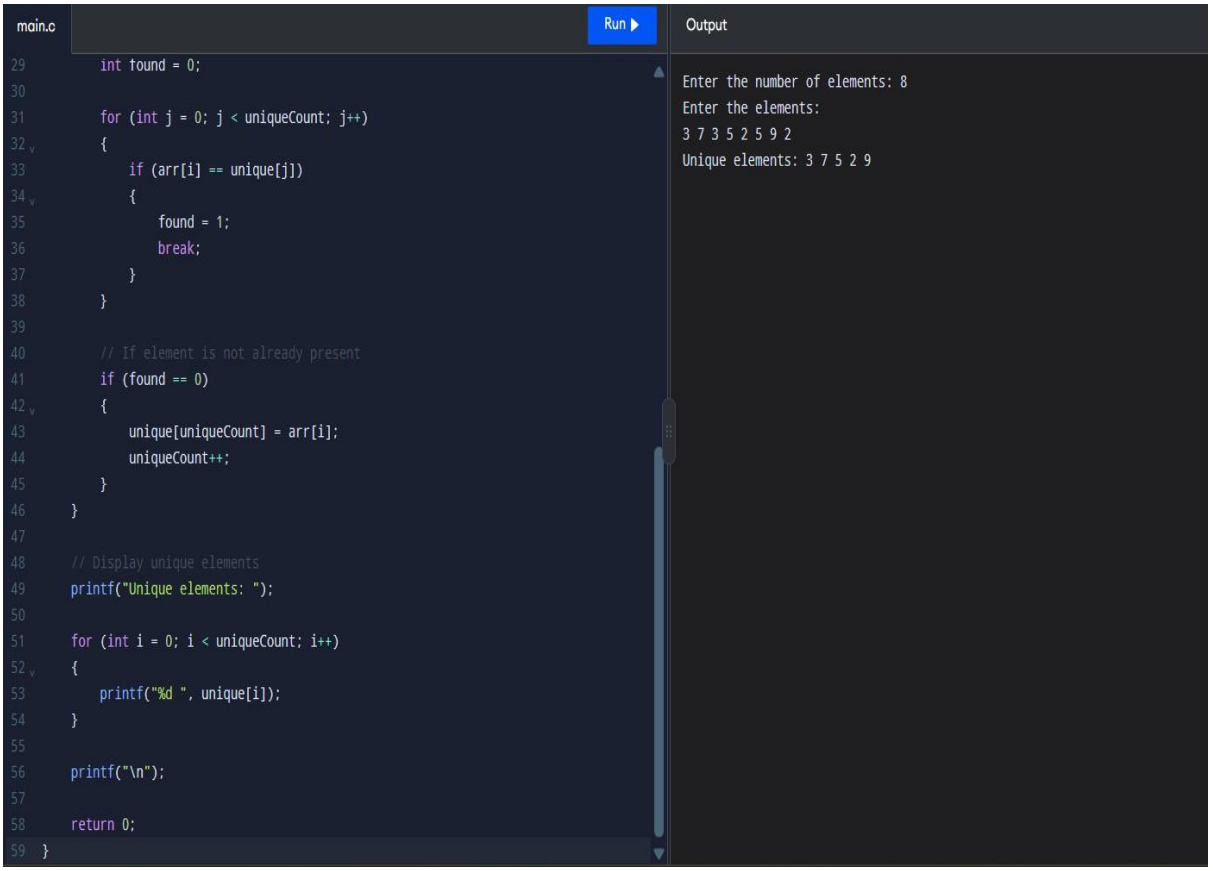
Expected Output: [-1, 2, 3, -2] (Order may vary)

List with Large Numbers

Input: [1000000, 999999, 1000000]

Expected Output: [1000000, 999999]

OUTPUT:



8. Sort an array of integers using the bubble sort technique. Analyze its time complexity using Big-O notation. Write the code

OUTPUT:

main.c

Run

```c
#include <stdio.h>

int main()
{
    int n, i, j, temp;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter the elements:\n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

    // Bubble Sort
    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
```

Output

```
Enter the number of elements: 5
Enter the elements:
64 34 25 12 22
Sorted array: 12 22 25 34 64
```

9. Checks if a given number x exists in a sorted array arr using binary search. Analyze its time complexity using Big-O notation.

Test Case:

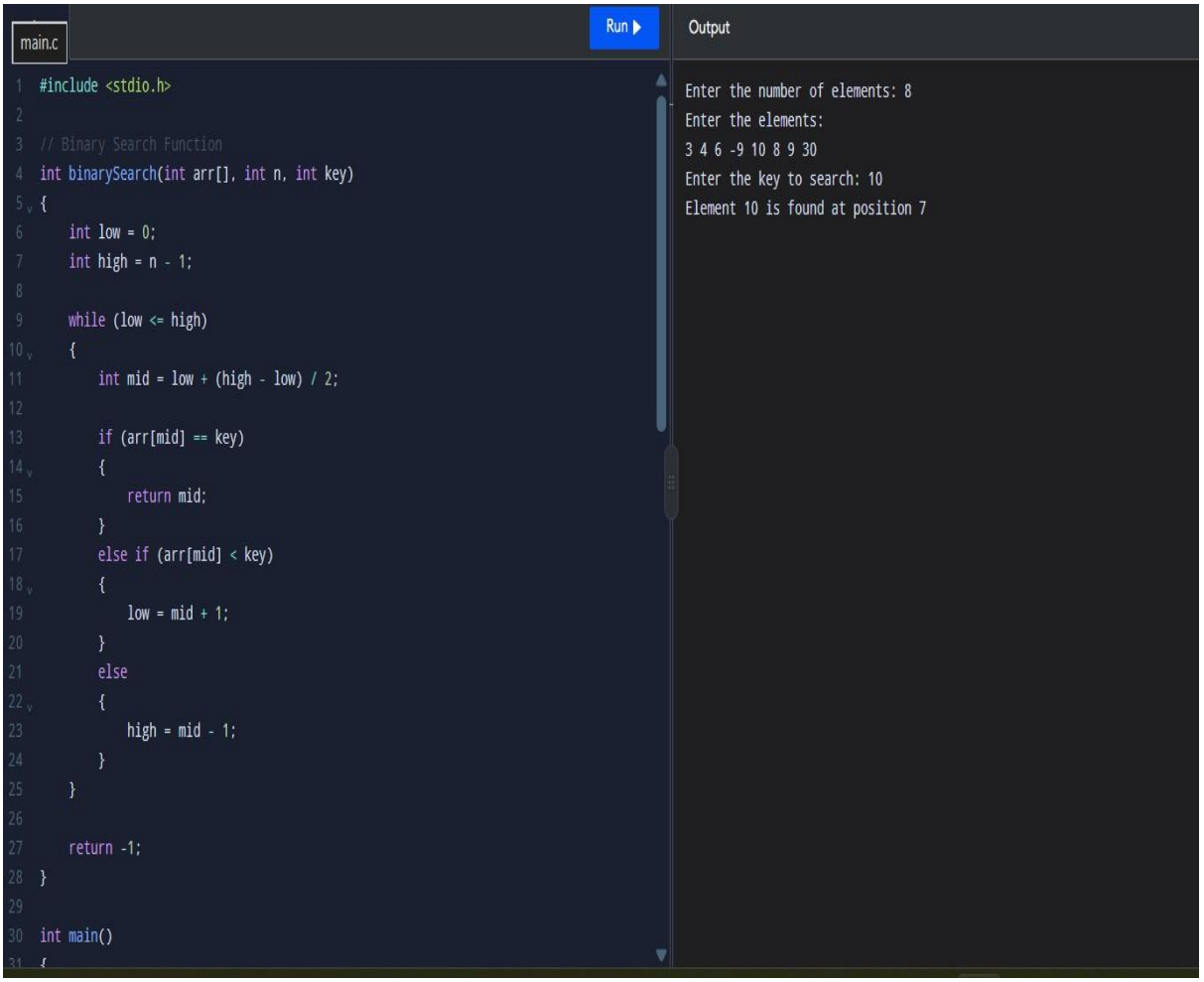
Example X={ 3,4,6,-9,10,8,9,30} KEY=10

Output: Element 10 is found at position 5

Example X={ 3,4,6,-9,10,8,9,30} KEY=100

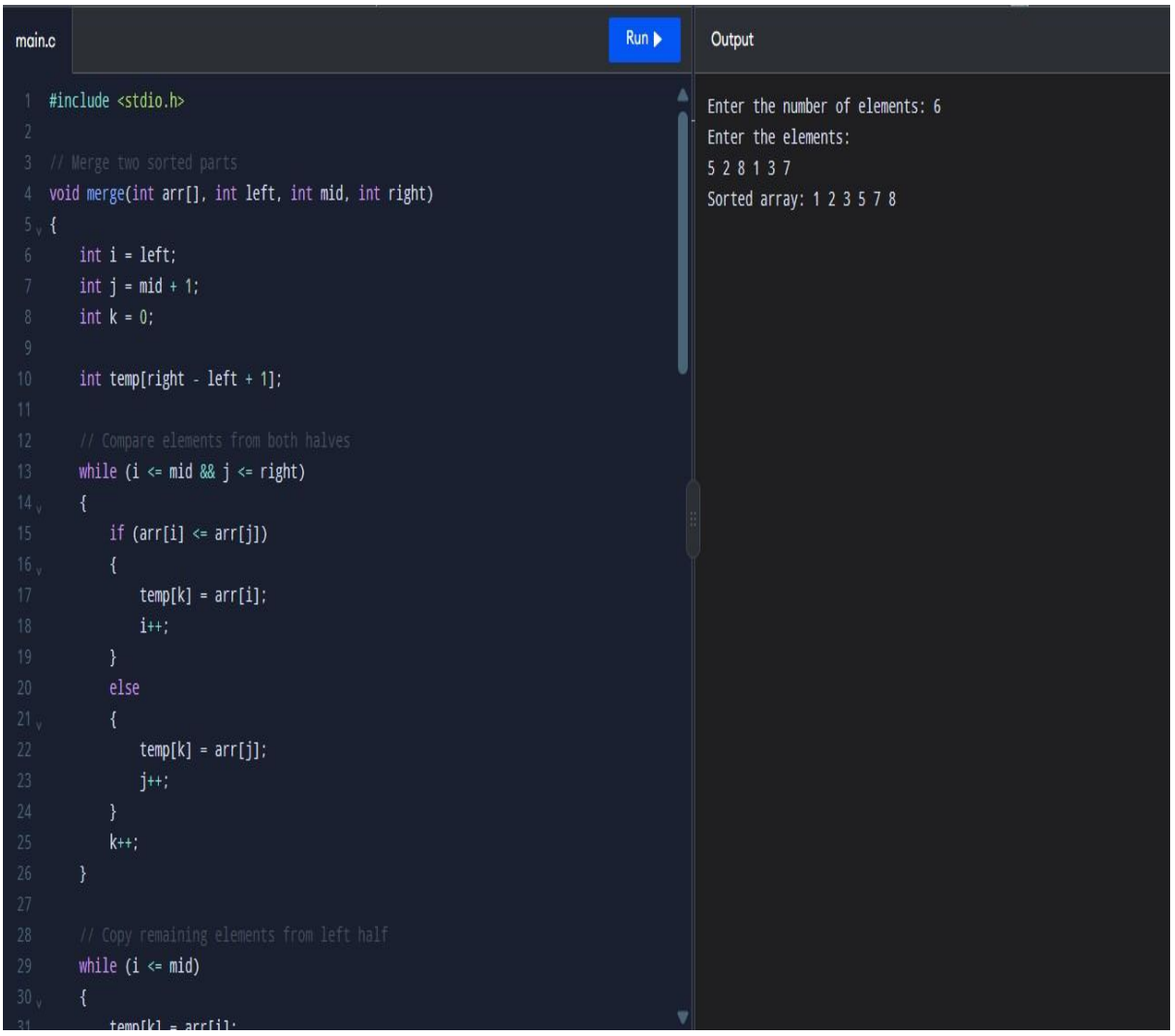
Output : Element 100 is not found

OUTPUT:



10. Given an array of integers nums, sort the array in ascending order and return it. You must solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

OUTPUT:


main.c
Run
Output
#include <stdio.h>
// Merge two sorted parts
void merge(int arr[], int left, int mid, int right)
{
int i = left;
int j = mid + 1;
int k = 0;
int temp[right - left + 1];
// Compare elements from both halves
while (i <= mid && j <= right)
{
if (arr[i] <= arr[j])
{
temp[k] = arr[i];
i++;
}
else
{
temp[k] = arr[j];
j++;
}
k++;
}
// Copy remaining elements from left half
while (i <= mid)
{
Enter the number of elements: 6
Enter the elements:
5 2 8 1 3 7
Sorted array: 1 2 3 5 7 8